

PROGRAMMAZIONE CONCORRENTE

by Cardinaletti Nicola



INDICE

COS'È LA PROGRAMMAZIONE CONCORRENTE?	3
PROBLEMATICHE DELLA PROGRAMMAZIONE CONCORRENTE	4
PROBLEMI CLASSICI DELLA PROGRAMMAZIONE CONCORRENTE	7
• Produttore e Consumatore.....	7
• Lettori e scrittori.....	11
• Filosofi a cena.....	14
• Barbieri sonnolento.....	18

COS'È LA PROGRAMMAZIONE CONCORRENTE?

La programmazione concorrente è un paradigma di programmazione in cui più attività (o processi/thread) vengono eseguite in modo contemporaneo, idealmente allo stesso tempo. Non significa necessariamente che vengano eseguite simultaneamente, ma che sono “in corso” nello stesso intervallo di tempo.

Il concetto di concorrenza è contrapposto a quello di sequenzialità. In cui in un sistema sequenziale i processi vengono eseguiti uno per volta e non si verifica alcuna forma di interazione tra essi durante l'esecuzione.

Dato che in un sistema concorrente la varietà di interazioni che si possono verificare tra processi in esecuzione è notevole (ovvero i processi possono condividere stesse zone di memoria/risorse), sono stati ideati problemi ipotetici, in merito alla gestione delle problematiche connesse alla concorrenza. Sulla base di queste teorie sono state sviluppate sia delle tecniche di programmazione (programmazione concorrente) sia dei linguaggi che integrano nativamente primitive per la corretta gestione della concorrenza.

PROBLEMATICHE DELLA PROGRAMMAZIONE CONCORRENTE

La concorrenza può portare ad una serie di problematiche legate all'utilizzo di una stessa risorsa condivisa da parte di più processi. Un determinato susseguirsi di eventi può causare condizioni critiche. La programmazione concorrente sfrutta alcuni principi per fronteggiare e risolvere questo tipo di problematiche.

Corse critiche (race condition)

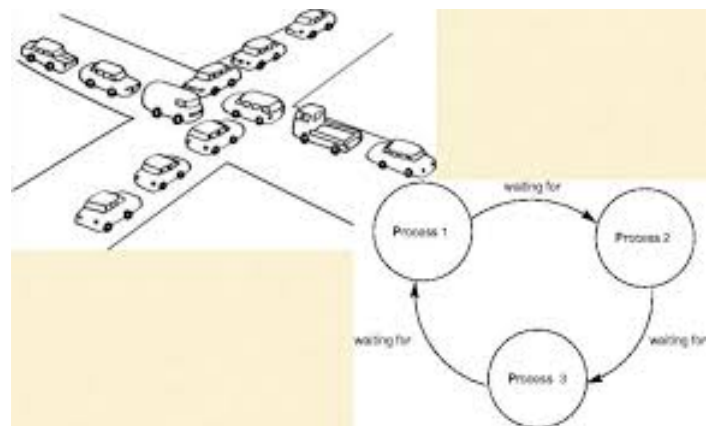
In alcuni sistemi può accadere che i processi in esecuzione condividano una risorsa comune. In particolare se si verifica che il risultato finale dell'esecuzione di più processi dipende dall'ordine in cui essi vengono eseguiti, questa è una condizione di corsa critica (race condition). Il risultato dell'esecuzione nel caso di corse critiche è assolutamente imprevedibile.

Il problema delle "corse critiche" può essere evitato impedendo che più di un processo per volta acceda a risorse condivise. Con la mutua esclusione si evita che più processi che contendono una risorsa riescano ad accedervi contemporaneamente.

Stallo (deadlock)

Quando ad un processo viene garantito l'accesso esclusivo (ad esempio tramite una mutua esclusione) ad una risorsa, possono crearsi situazioni di stallo.

Formalmente un insieme di processi è in stallo (deadlock) quando ogni processo dell'insieme attende un evento che può avvenire soltanto tramite un altro processo dell'insieme. Essendo tutti i processi in attesa, nessuno potrà mai creare l'evento di sblocco protraendo l'attesa all'infinito. Le tecniche per individuare situazioni di stallo prevedono l'analisi di grafi delle risorse allocate oppure mediante la creazione di cosiddette "matrici di allocazione". La risoluzione degli stalli può essere affrontata in vari modi. Concettualmente si possono suddividere in:



Risoluzione mediante prerilascio: viene scelto un processo che detiene una risorsa dall'insieme dei processi in stallo e viene tolto l'accesso esclusivo (prerilascio o preemption) ad una risorsa condivisa (causa di stallo).

L'operazione è talvolta difficile, talvolta impossibile e dipende dal tipo di risorsa che il processo stava bloccando.

Risoluzione mediante punto di controllo: vengono creati dei registri (checkpoint) che descrivono lo stato di utilizzo delle risorse condivise. Quando viene rilevato uno stallo si effettua un "ritorno" (rollback to checkpoint) alle condizioni precedenti. Anche questa tecnica è di difficile o addirittura impossibile realizzazione poiché il ritorno potrebbe causare perdita o inconsistenza di dati.

Risoluzione mediante eliminazione: viene scelto un processo e viene terminato. Questa tecnica è anch'essa molto complessa e richiede di fare stime e assunzioni sul tipo di processo da eliminare. Inoltre non è garantita l'uscita dalla condizione di stallo per cui potrebbe essere necessario terminare altri processi, situazione che complica ulteriormente la problematica.

È anche possibile effettuare delle stime sulle risorse che verranno impegnate da un processo. Grazie a tali stime, esistono sistemi in cui invece di risolvere le situazioni di stallo risulta più semplice evitarle a priori.

Tutte le tecniche prevedono la costruzione di matrici che tengono traccia dell'utilizzo delle risorse (matrici di traiettoria di risorse) o si basano su algoritmi noti come l'algoritmo del banchiere.

Starvation

Letteralmente inedia, è un problema in stretta relazione con lo stallo. Quando le politiche di assegnazione delle risorse condivise favoriscono un processo rispetto ad un altro, il processo a cui vengono assegnate in minor misura soffre di starvation. Esso è infatti bloccato e non può proseguire sebbene non si trovi in una condizione di stallo. Nei sistemi in cui si accede a risorse condivise è sempre necessario stabilire una politica per le priorità e l'ordine



con cui esse vengono ripartite. Sebbene queste politiche possano risultare quanto più eque, esse possono portare a condizioni di starvation.

PROBLEMI CLASSICI DELLA PROGRAMMAZIONE CONCORRENTE

I problemi classici della programmazione concorrente sono esempi teorici (ma molto realistici) usati per studiare, insegnare e verificare i meccanismi di sincronizzazione tra processi o thread che accedono a risorse condivise. I problemi che vedremo sono:

- Produttore e Consumatore

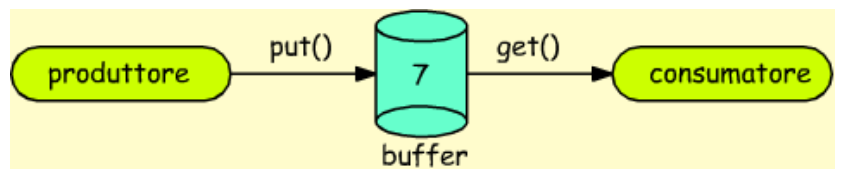
In informatica, il problema del produttore-consumatore (conosciuto anche con il nome di problema del

buffer limitato o

bounded-buffer problem) è un

esempio classico di

sincronizzazione tra processi.



Il problema descrive due processi, uno produttore ed uno consumatore, che condividono un buffer comune, di dimensione fissata. Compito del produttore è generare dati e depositarli nel buffer in continuo. Contemporaneamente, il consumatore utilizzerà i dati prodotti, rimuovendoli di volta in volta dal buffer. Il problema è assicurare che il produttore non elabori nuovi dati se il buffer è pieno, e che il consumatore non cerchi dati se il buffer è vuoto.

La soluzione per il produttore è sospendere la propria esecuzione se il buffer è pieno; non appena il consumatore avrà prelevato un elemento dal buffer, esso "sveglierà" il produttore, che ricomincerà quindi a riempire il buffer. Allo stesso modo, il consumatore si sospenderà se il buffer è vuoto; non appena il produttore avrà scaricato dati nel buffer, risveglierà il consumatore. Questa soluzione può essere implementata tramite delle strategie di comunicazione tra processi, tipicamente con dei semafori. Una soluzione errata potrebbe dar luogo ad un deadlock, in cui entrambi i processi aspettano di essere risvegliati.

Esempio di codice risolutivo, in linguaggio java:

```
public class Buffer {

    private final int[] buffer;

    private int count = 0;

    private int in = 0;

    private int out = 0;

    public Buffer(int size) {

        buffer = new int[size];

    }

    public synchronized void put(int value) throws InterruptedException {

        while (count == buffer.length) {

            wait(); // buffer pieno

        }

        buffer[in] = value;

        in = (in + 1) % buffer.length;

        count++;

        notifyAll(); // sveglia i consumatori

    }

    public synchronized int get() throws InterruptedException {

        while (count == 0) {

            wait(); // buffer vuoto

        }

        int value = buffer[out];

        out = (out + 1) % buffer.length;

        count--;

        notifyAll(); // sveglia i produttori

        return value;

    }

}
```

```
}
```

```
public class Produttore extends Thread {  
  
    private final Buffer buffer;  
  
    public Produttore(Buffer buffer) {  
  
        this.buffer = buffer;  
  
    }  
  
    @Override  
    public void run() {  
  
        try {  
  
            for (int i = 1; i <= 10; i++) {  
  
                buffer.put(i);  
  
                System.out.println("Prodotto: " + i);  
  
                Thread.sleep(500);  
  
            }  
  
        } catch (InterruptedException e) {  
  
            Thread.currentThread().interrupt();  
  
        }  
  
    }  
}
```

```
public class Consumatore extends Thread {  
  
    private final Buffer buffer;  
  
    public Consumatore(Buffer buffer) {  
  
        this.buffer = buffer;  
  
    }  
  
    @Override  
    public void run() {  
  
        try {
```

```

        for (int i = 1; i <= 10; i++) {

            int value = buffer.get();

            System.out.println("Consumata: " + value);

            Thread.sleep(800);

        }

    } catch (InterruptedException e) {

        Thread.currentThread().interrupt();

    }

}

}

```

```

public class Main {

    public static void main(String[] args) {

        Buffer buffer = new Buffer(5);

        Produttore p = new Produttore(buffer);

        Consumatore c = new Consumatore(buffer);

        p.start();

        c.start();

    }

}

```

- Lettori e scrittori

In informatica il problema dei Lettori e scrittori, modella l'accesso a basi di dati; ovvero: Su una base di dati agiscono scrittori, che ne modificano il contenuto, e lettori che ne recuperano il contenuto. Una corretta programmazione concorrente prevede l'accesso di un solo scrittore in fase di modifica (e di nessun lettore onde evitare problemi), mentre l'accesso di tanti lettori è possibile a patto che non ci siano scrittori attivi contemporaneamente.



Esempio di codice risolutivo, in linguaggio java:

```
import java.util.concurrent.Semaphore;

public class LettoriScrittori {

    // Numero di lettori attivi

    private static int lettori = 0;

    // Semafori

    private static Semaphore mutex = new Semaphore(1);    // protegge "lettori"

    private static Semaphore scrittura = new Semaphore(1); // accesso alla risorsa

    // Classe Lettore

    static class Lettore extends Thread {

        private int id;

        public Lettore(int id) {

            this.id = id;
```

```
}
```

```
@Override
```

```
public void run() {
```

```
    try {
```

```
        // Sezione di ingresso
```

```
        mutex.acquire();
```

```
        lettori++;
```

```
        if (lettori == 1) {
```

```
            scrittura.acquire(); // primo lettore blocca gli scrittori
```

```
        }
```

```
        mutex.release();
```

```
        // Sezione critica (lettura)
```

```
        System.out.println("Lettore " + id + " sta LEGGENDO");
```

```
        Thread.sleep(1000);
```

```
        // Sezione di uscita
```

```
        mutex.acquire();
```

```
        lettori--;
```

```
        if (lettori == 0) {
```

```
            scrittura.release(); // ultimo lettore sblocca gli scrittori
```

```
        }
```

```
        mutex.release();
```

```
    } catch (InterruptedException e) {
```

```
        e.printStackTrace();
```

```
    }
```

```
}
```

```
}
```

```
// Classe Scrittore

static class Scrittore extends Thread {

    private int id;

    public Scrittore(int id) {

        this.id = id;

    }

    @Override

    public void run() {

        try {

            scrittura.acquire();

            // Sezione critica (scrittura)

            System.out.println("Scrittore " + id + " sta SCRIVENDO");

            Thread.sleep(1500);

            scrittura.release();

        } catch (InterruptedException e) {

            e.printStackTrace();

        }

    }

}
```

```
// Main

public static void main(String[] args) {

    for (int i = 1; i <= 5; i++) {

        new Lettore(i).start();

    }

    for (int i = 1; i <= 2; i++) {

        new Scrittore(i).start();

    }

}
```

- Filosofi a cena

Il problema dei filosofi a cena (Dining Philosophers Problem) è un classico esempio di problema di concorrenza in informatica, introdotto da Edsger W. Dijkstra nel 1965.

Viene utilizzato per spiegare e analizzare le difficoltà legate alla gestione delle risorse condivise, alla sincronizzazione dei processi e ai problemi di deadlock e starvation nei sistemi operativi.



Il problema è descritto in modo astratto attraverso una metafora:

Ci sono cinque filosofi seduti attorno a un tavolo rotondo, davanti a ogni filosofo c'è un piatto di spaghetti, tra ogni coppia di filosofi c'è una sola forchetta (in totale cinque forchette).

Per mangiare, un filosofo ha bisogno di due forchette: quella alla sua sinistra e quella alla sua destra.

I problemi principali in questo problema sono 3:

1. Deadlock (stallo)

Il deadlock si verifica quando ogni filosofo prende una forchetta e aspetta indefinitamente l'altra, che è occupata da un vicino.

In questa situazione nessun filosofo può continuare a mangiare e il sistema rimane bloccato.

2. Starvation (attesa indefinita)

La starvation si verifica quando uno o più filosofi non riescono mai a ottenere entrambe le forchette, perché altri filosofi mangiano continuamente.

Il sistema funziona, ma non in modo equo.

3. Mutua esclusione

Le forchette rappresentano risorse che possono essere usate da un solo filosofo alla volta, quindi devono essere protette da meccanismi di mutua esclusione.

Esempio di codice risolutivo, in linguaggio java:

```
import java.util.concurrent.Semaphore;

class Filosofo extends Thread {

    private int id;

    private Semaphore sinistra;

    private Semaphore destra;

    private Semaphore cameriere;

    public Filosofo(int id, Semaphore sinistra, Semaphore destra, Semaphore cameriere) {

        this.id = id;

        this.sinistra = sinistra;

        this.destra = destra;

        this.cameriere = cameriere;

    }

    private void pensa() throws InterruptedException {

        System.out.println("Filosofo " + id + " sta pensando.");

        Thread.sleep((int)(Math.random() * 1000));

    }

    private void mangia() throws InterruptedException {

        System.out.println("Filosofo " + id + " sta mangiando.");

        Thread.sleep((int)(Math.random() * 1000));

    }

    @Override

    public void run() {
```

```

try {

    while (true) {

        pensa();

        // Chiede il permesso al cameriere

        cameriere.acquire();

        // Prende le forchette

        sinistra.acquire();

        destra.acquire();

        mangia();

        // Rilascia le forchette

        sinistra.release();

        destra.release();

        // Avvisa il cameriere

        cameriere.release();

    }

} catch (InterruptedException e) {

    e.printStackTrace();

}

}

}

```

```

public class FilosofiACena {

    public static void main(String[] args) {

        int N = 5;

        Semaphore[] forchette = new Semaphore[N];

        for (int i = 0; i < N; i++) {

            forchette[i] = new Semaphore(1);

        }

        // Cameriere: massimo N-1 filosofi possono mangiare
    }

}

```

```
Semaphore cameriere = new Semaphore(N - 1);

Filosofo[] filosofi = new Filosofo[N];

for (int i = 0; i < N; i++) {

    Semaphore sinistra = forchette[i];

    Semaphore destra = forchette[(i + 1) % N];

    filosofi[i] = new Filosofo(i, sinistra, destra, cameriere);

    filosofi[i].start();

}

}

}
```

● Barbiere sonnolento

Formulato per la prima volta da Edsger W. Dijkstra, questo problema serve a illustrare le difficoltà della concorrenza, ovvero: accesso a risorse condivise, sincronizzazione tra produttori e consumatori, gestione dell'attesa e del risveglio dei processi.

Immaginiamo una barberia con un solo barbiere ed una sola sedia da barbiere dove può tagliare i capelli del cliente e N sedie nella sala d'attesa. I clienti arrivano in modo imprevedibile.



Regole:

1. Se non ci sono clienti, il barbiere dorme
2. Se arriva un cliente:

Se il barbiere dorme lo sveglia

Se il barbiere è occupato:

Se c'è una sedia libera il cliente attende

Se non c'è posto il cliente se ne va

3. Il barbiere serve un cliente alla volta

L'obiettivo è coordinare correttamente il thread del barbiere e i thread dei clienti evitando che si creino casi di: race condition, deadlock, busy waiting, perdita di clienti o risvegli inutili.

Esempio di codice risolutivo, in linguaggio java:

```
public class Main {

    public static void main(String[] args) {

        int numeroSedie = 3;

        Barberia barberia = new Barberia(numeroSedie);

        Barbiere barbiere = new Barbiere(barberia);

        barbiere.start();

        for (int i = 1; i <= 10; i++) {

            new Cliente(barberia, i).start();

            try {

                Thread.sleep(1000); // arrivo clienti a intervalli

            } catch (InterruptedException e) {

                e.printStackTrace();

            }

        }

    }

}

-----

import java.util.concurrent.Semaphore;

public class Barberia {

    int sedieLibere;

    Semaphore clienti = new Semaphore(0);

    Semaphore barbiere = new Semaphore(0);

    Semaphore mutex = new Semaphore(1);

    public Barberia(int numeroSedie) {

        this.sedieLibere = numeroSedie;

    }

}
```

```
public class Barbiere extends Thread {

    private Barberia barberia;

    public Barbiere(Barberia barberia) {

        this.barberia = barberia;

    }

    public void run() {

        try {

            while (true) {

                barberia.clienti.acquire();    // attende un cliente

                barberia.mutex.acquire();

                barberia.sedieLibere++;        // libera una sedia

                barberia.barbiere.release();    // pronto a servire

                barberia.mutex.release();

                tagliaCapelli();

            }

        } catch (InterruptedException e) {

            e.printStackTrace();

        }

    }

    private void tagliaCapelli() throws InterruptedException {

        System.out.println("Il barbiere sta tagliando i capelli...");

        Thread.sleep(3000);

        System.out.println("Il barbiere ha finito.");

    }

}
```

```

public class Cliente extends Thread {

    private Barberia barberia;

    private int id;

    public Cliente(Barberia barberia, int id) {

        this.barberia = barberia;

        this.id = id;

    }

    public void run() {

        try {

            barberia.mutex.acquire();

            if (barberia.sedieLibere > 0) {

                barberia.sedieLibere--;

                System.out.println("Cliente " + id + " si siede. Sedie libere: " + barberia.sedieLibere);

                barberia.clienti.release(); // segnala la presenza

                barberia.mutex.release();

                barberia.barbiere.acquire(); // attende il barbiere

                fattiTagliare();

            } else {

                System.out.println("Cliente " + id + " se ne va (nessuna sedia libera)");

                barberia.mutex.release();

            }

        } catch (InterruptedException e) {

            e.printStackTrace();

        }

    }

    private void fattiTagliare() throws InterruptedException {

        System.out.println("Cliente " + id + " viene servito.");

    }

}

```